Simulation Methods

BESSON Adrien

Contents

1 Ratio of Uniforms **2**

2 Importance Sampling **5**

3 Acceptance-Rejection **8**

4 Quasi-Monte Carlo **14**

5 Some Data **18**

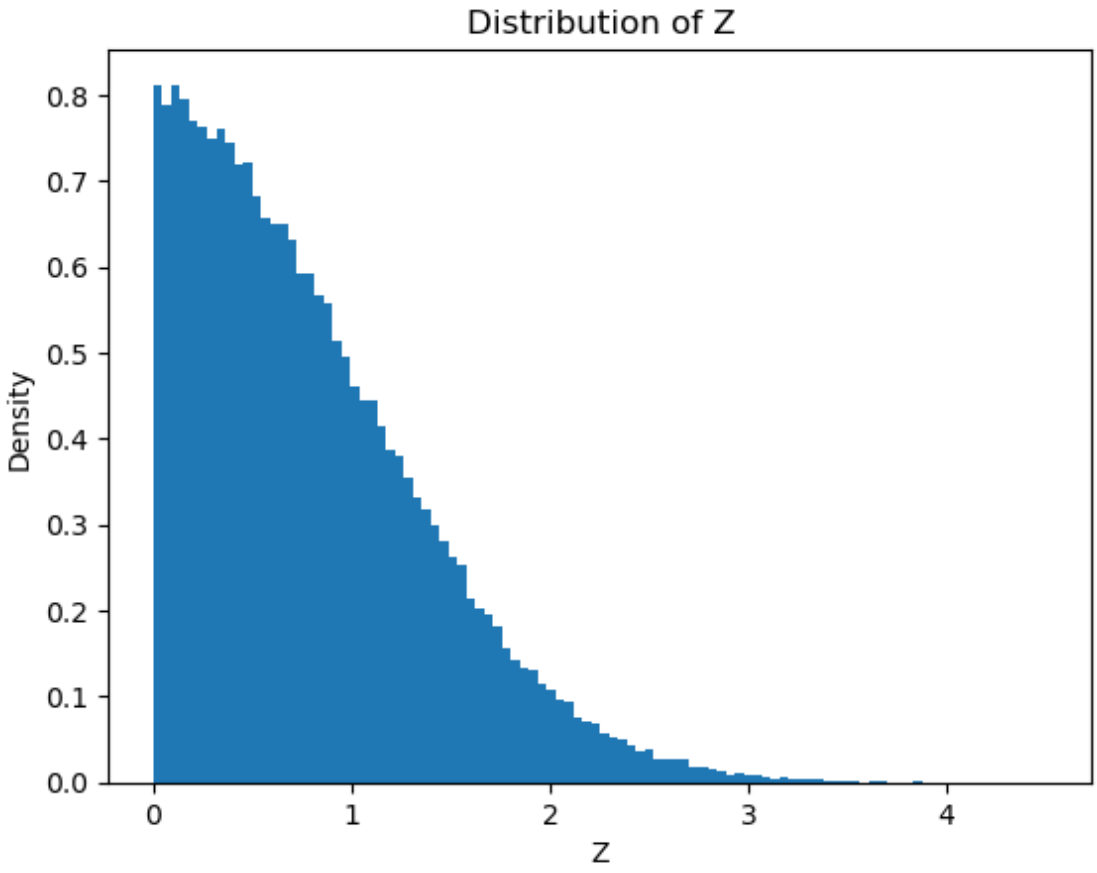
1 Ratio of Uniforms

1. Implement the algorithm, what does the distribution of Z look like?

So we start by implementing the algorithm with a while loop, and we only keep the U functions that respects the condition $U_1 \leq e^{(-\frac{1}{4}(\frac{U_2}{U_1})^2)}$

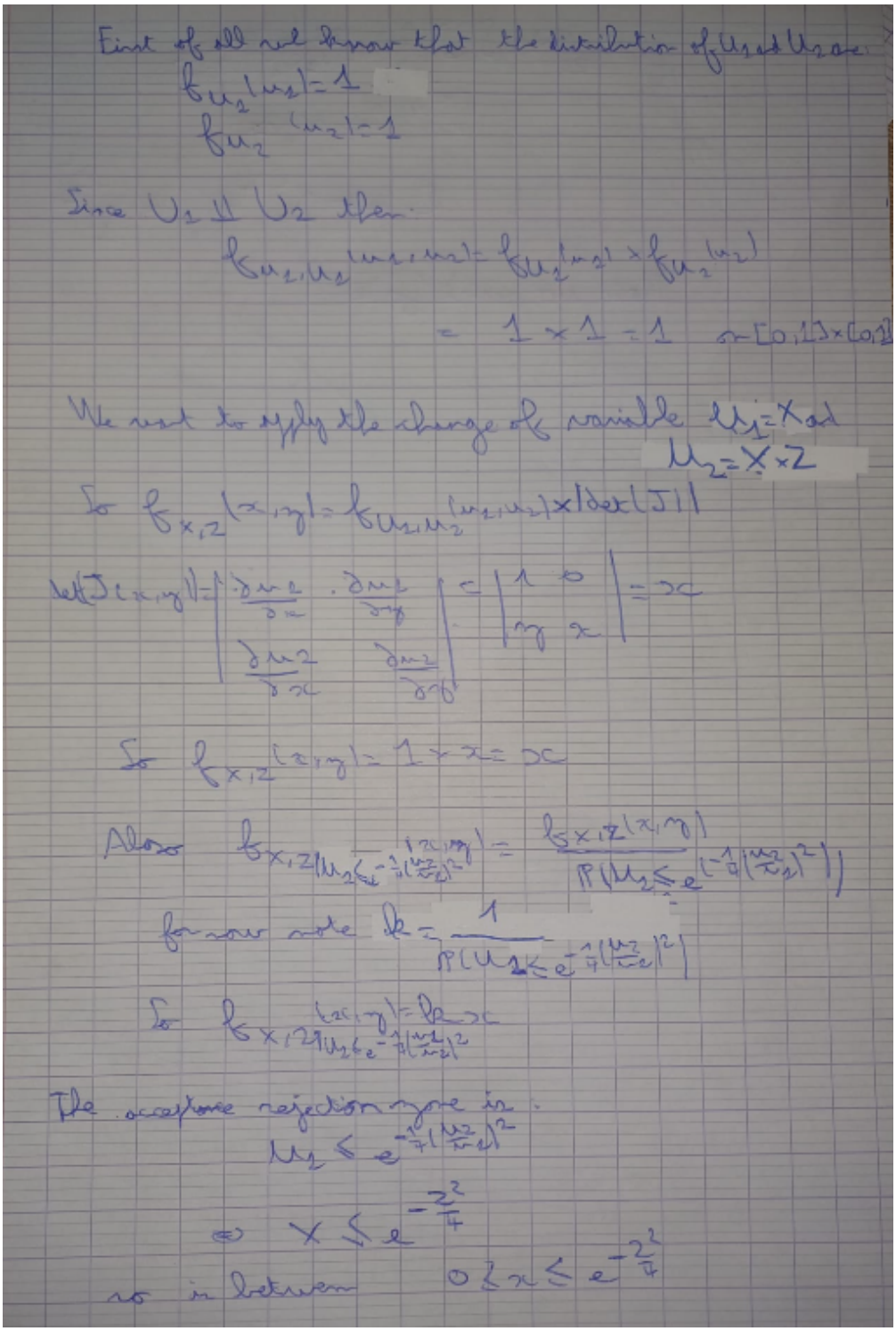
```
n=100000
sim=[]
while(len(sim)<n):
    u1=np.random.uniform(0, 1)
    u2=np.random.uniform(0, 1)
    if(u1 <= np.exp(−0.25∗(u2/u1)∗∗2)):
        Z = u2/u1
        sim.append(Z)

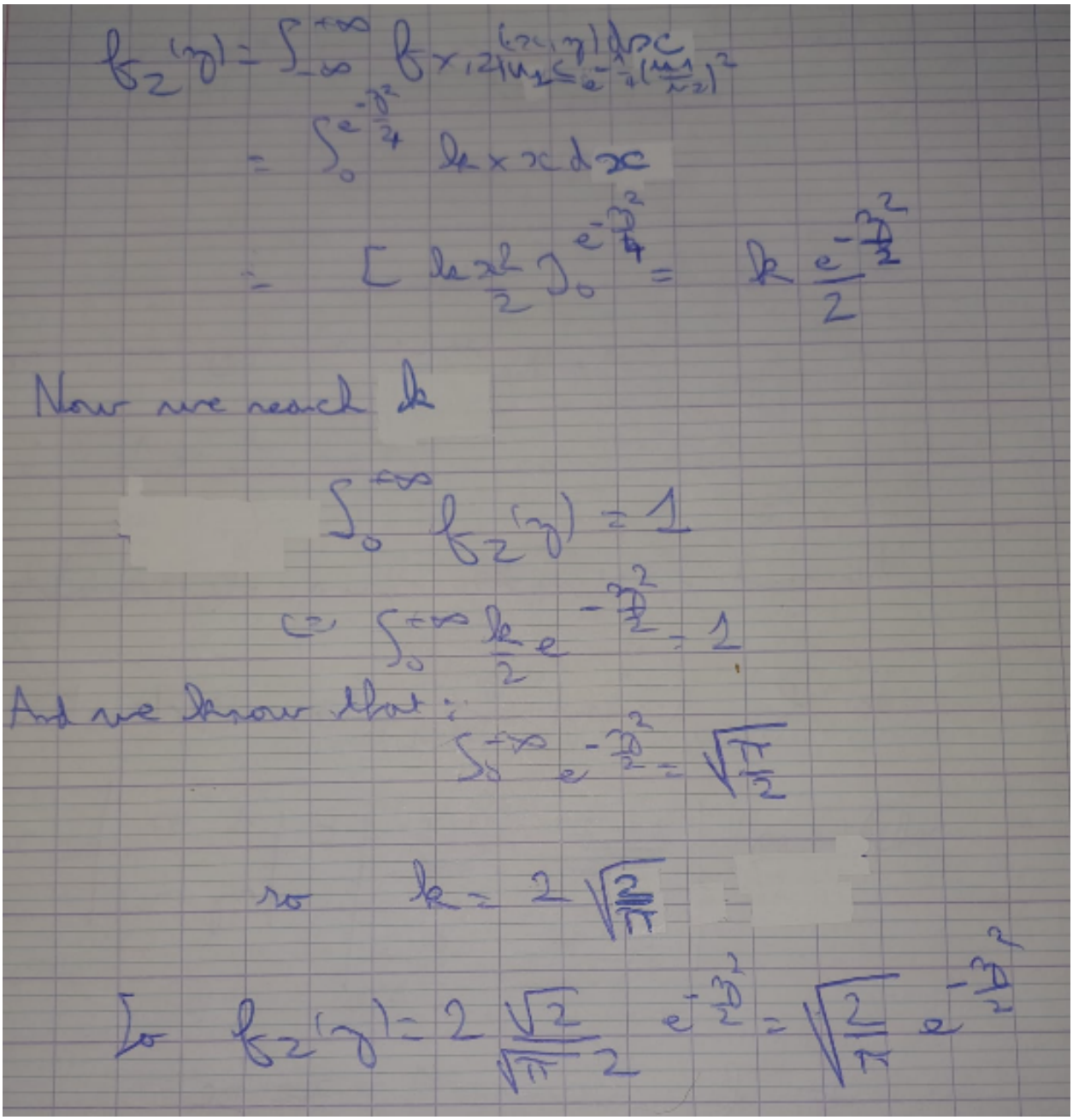
plt.hist(sim, bins=100, density=True)
plt.title("Distribution of Z")
plt.xlabel("Z")
plt.ylabel("Density")
plt.show()
```



The distribution of Z looks like right (positive) part of the standard Normal law distribution, N(0,1). But it is scaled since it goes up to 0.8 at z=0, which is the double of a standard Normal law distribution N(0,1). So basically it is a Half-normal distribution.

2. Find the distribution of Z by using the change of variables U1 = X, U2 = XZ.





=

Like we expected from looking at the graph, this does give us a Half standard normal Law. Let N follow a N(0,1) then we have $X = |N|$ follow a half standard normal law which is the same law as Z.

3. What is the theoretical acceptance rate? What is the theoretical average number of samples of (U1, U2) needed to generate a sample Z?

We said that

$$k = \frac{1}{P(U_1 \leq e^{-\frac{1}{4}(\frac{u2}{u1})^2})}$$

So the probability of acceptance rate is given by :

$$P(U_1 \leq e^{-\frac{1}{4}(\frac{u2}{u1})^2}) = \frac{1}{k} = \frac{\sqrt{\pi}}{2\sqrt{2}} = 0.6266 = 62.66\%$$

When doing the acceptance rejection method, what we do is we try to generate once, then if it fail we try again, this is exactly what a Geometric law would be like. Try until you make it. And so the expected value of the Geometric law will be the theorical averages number of sample U1,U2, needed to generate a sample Z. Also this confirms with the course that says that the acceptance probability is 1/M (here k) and that the expected number of iterations is M (here k).

$$theoretical\ average\ number\ of\ samples = \frac{1}{P(U_1 \leq e^{-\frac{1}{4}(\frac{u2}{u1})^2})} = k = 1.5957 \approx 1.6$$

To conclude on average the algorithm will generate about 1.6 pairs of U1,U2 to generate one sample Z.

We added a counter in the algorithm to see if we did find this theorical average number, and we did found an average of 1.6 like we expected from our calculations.

```
n=100000
count=0 # the new parameter to count for the total number of u1,u2
#pair generated to generate n sample Z
sample=[]
while(len(sample)<n):
    count+=1
    u1=np.random.uniform(0, 1)
    u2=np.random.uniform(0, 1)
    if(u1 <= np.exp(-0.25*(u2/u1)**2)):
        Z = u2/u1
        sample.append(Z)
```

2 Importance Sampling

Question 1 : Write the solution using closed formula (Black-Scholes). What happens when you evaluate this formula?

Black-Scholes Formula for an European Option :

$$C = S_0 * N(d_1) - K * e^{-rT} * N(d_2)$$

For a Digital Call Option :

$$D = e^{-rT} * N(d_2)$$

$$d_1 = \frac{ln(S_0/K) + (r + \sigma^2/2)T}{\sigma\sqrt{T}}$$

$$d_2 = d_1 - \sigma\sqrt{T} = \frac{ln(S_0/K) + (r - \sigma^2/2)T}{\sigma\sqrt{T}}$$

To find how many Digital calls are worth a European Call we will need to look at the ratio C/D

```
S0=53.6
r=0.015
sigma=0.235

T=1
K=650000

d1=(np.log(S0/K)+(r+sigma**2/2)*T)/(sigma*np.sqrt(T))
d2=d1-sigma*np.sqrt(T)

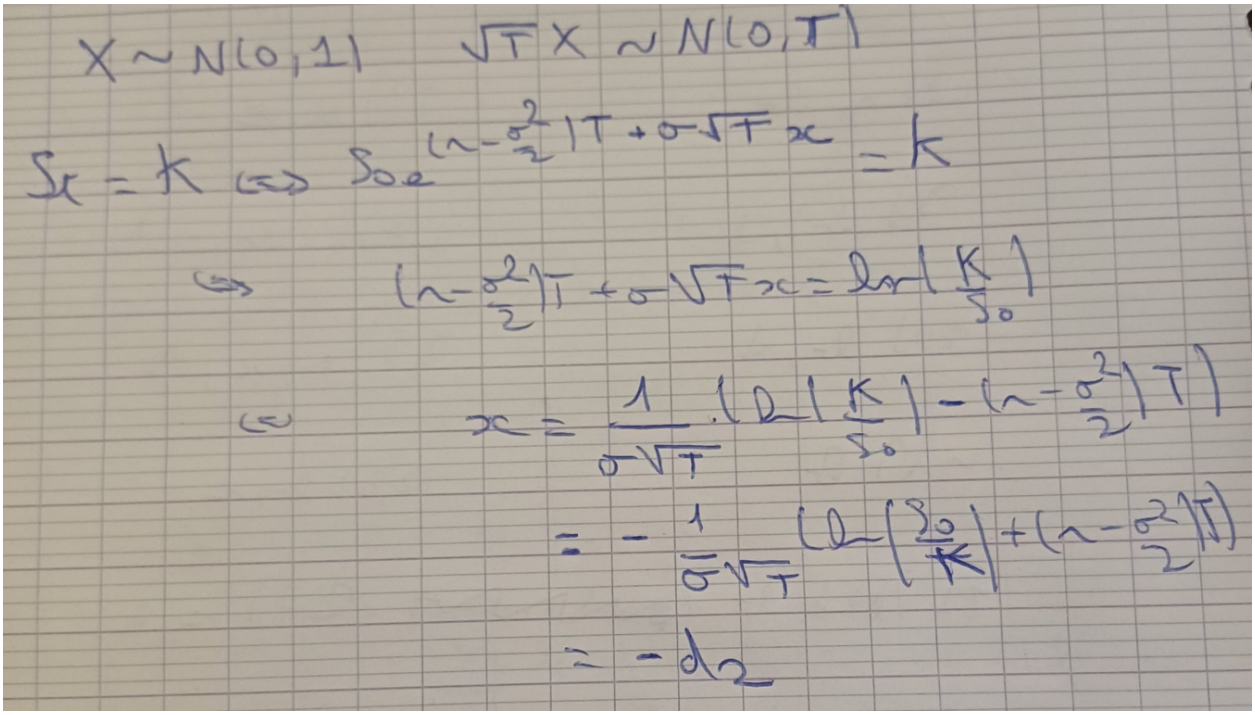
Nd1=norm.cdf(d1)
Nd2=norm.cdf(d2)

C=S0*Nd1-K*np.exp(-r*T)*Nd2
D=np.exp(-r*T)*Nd2
```

Since the d1 and d2 absolute values are very high, d1=-39,83 and d2=-40,067. When we take there values from the cumulative Normal distribution function N(0,1), the value is so small because too extreme that it is considered as 0. There absolute values are so high because the strike K=650000 compared to the spot price S_0=53.6, the option is basically worthless because it will never hit the strike, and so we get a probability of 0. But a price of 0 for an asset is of course never possible. The Black-Scholes model performs well in normal market conditions but does not account for extreme events. Because of this we get two 0 and so we can't do a ratio since 0/0 is an undefined number.

Question 2 :Apply the principle of importance sampling to estimate the ratio. Explain how the trick below is used.

The issue we had before is that we were far out of the money so we f couldn't reach K=650000, and so thanks to Importance Sampling the goal is to use a new density function g and use the Randon-Nikodin theorem to do a change of measure. First of all we need to find a drift μ, such that we use a new normal law G that follow a N(μ,1). To find μ, we put St=K and isolate x, that will be our μ and it will be equal to -d2 (see the demonstration below). The goal of this new normal law is to be able to be half the time in the money and half the time out of the money, instead of always out of the money.



Now we take the densities of the functions, for N(0,1), $f(x) = \frac{1}{\sqrt{2\pi}}e^{-\frac{x^2}{2}}$ and for the N(μ,1) $g(x) = \frac{1}{\sqrt{2\pi}}e^{-\frac{(x-\mu)^2}{2}}$ We now want to apply the theorem Randon Nikodin with the weight f/g. We calculate the ratio f/g and we get $\frac{f(x)}{g(x)} = e^{(\frac{-2\mu x+\mu^2}{2})}$. Finally we the term l_i that is the ln of this ratio and is equal to $l_i = \ln(\frac{f(x)}{g(x)}) = -\mu x + \frac{1}{2}\mu^2$ But if we keep l_i like that it will be too small to use exponantial of l_i in the final formula $\frac{\sum_i \exp(l_i)f_i}{\sum_i \exp(l_i)g_i}$. And since l_i will be too small it will give nan. For this we use a trick and multiply both the numerator and denominator by a constant term e^C, and since $e^C * e^{l_i} = e^{C+l_i}$. And so this final formula $\frac{\sum_i \exp(l_i)f_i}{\sum_i \exp(l_i)g_i}$ represent the sum of the term exponantial of $l_i + C$ times the payoff of a European Call divided by the same term $l_i + C$ times the payoff of a digital Call.

To generate the normal law we use a Box Muller function.

```
def Box_Muller(n):
    u1=np.random.rand(n)
    u2=np.random.rand(n)
    R=-2*np.log(u1)
    theta=2*np.pi*u2
    return np.sqrt(R)*np.cos(theta),np.sqrt(R)*np.sin(theta)
```

And then we do what we explained above

```
N = 100000
mu=-d2

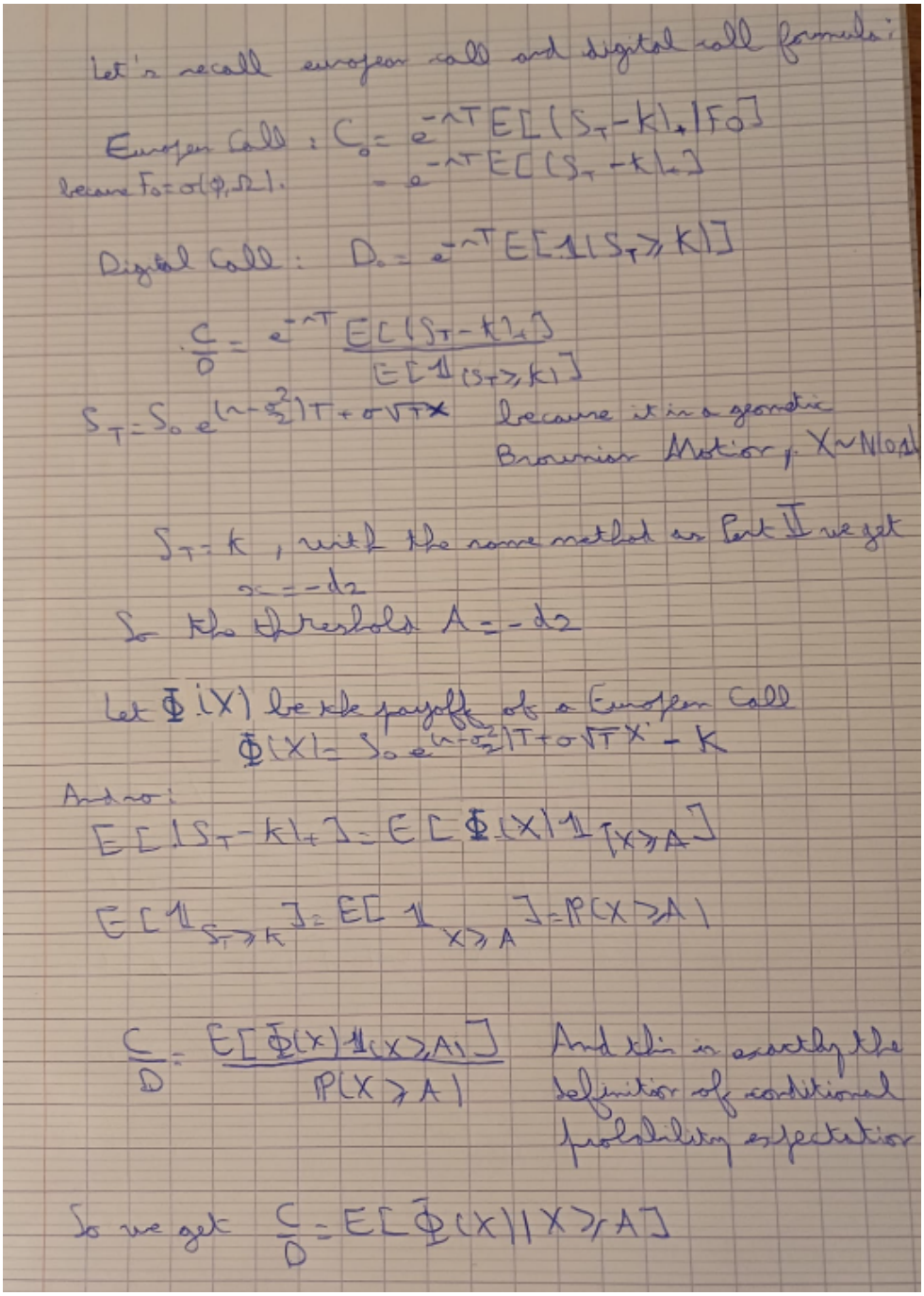
Z,_=Box_Muller(N)
X=mu+Z
St=S0*np.exp((r-(sigma**2)/2)*T+sigma*np.sqrt(T)*X)
payoff_C = np.maximum(St-K, 0)
payoff_D = (St>K).astype(float)

l_i = -mu*X+0.5*mu**2
C = 500 #this value is arbitrary and can be changed but need to be
#high enought to don't have exponantial of -745 and below or not
#too high too don't have a value that can't be computed.
ratio=np.sum(payoff_C*np.exp(l_i+C))/np.sum(payoff_D*np.exp(l_i+C))
```

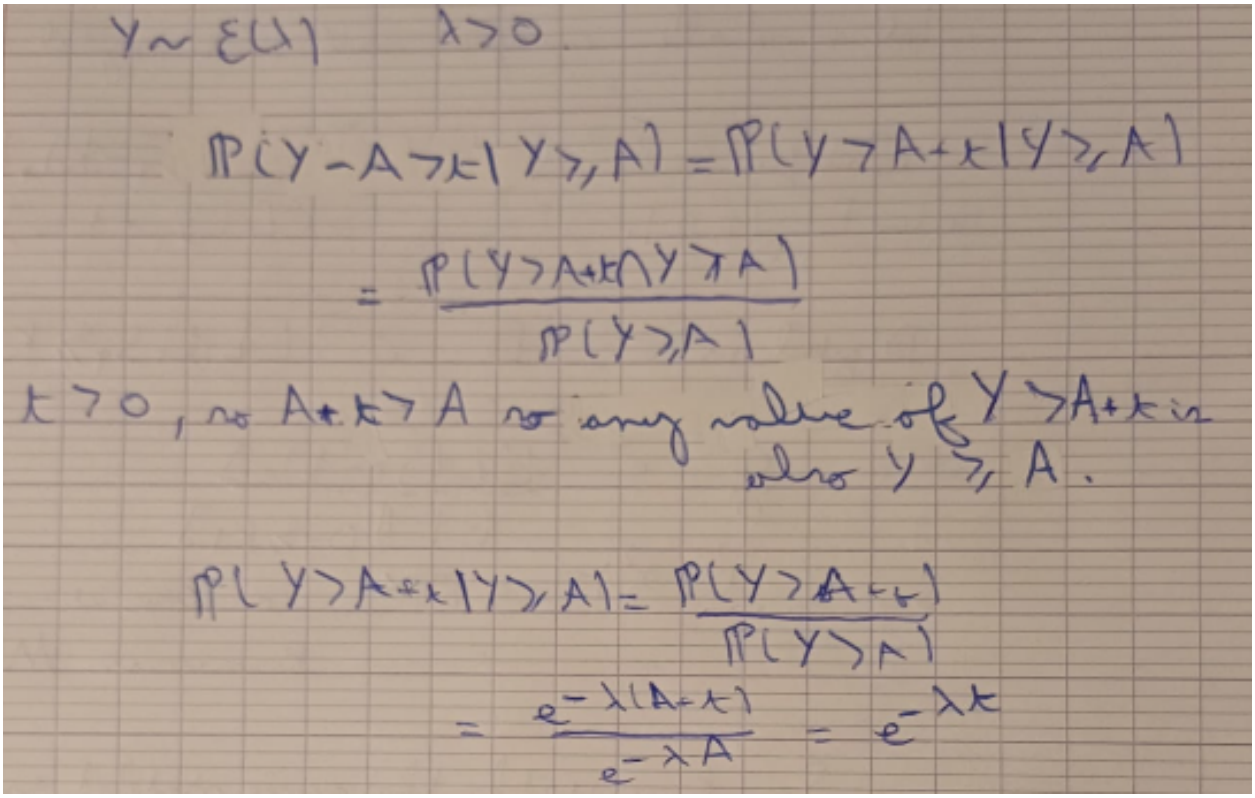
3. Is it realistic? why?
This simulation method give us a ratio between around 3800 and 3900. So one European Call with maturity T=1 and K=650000 is worth about 3800 more than a Digital European Call. This result make sense because a Digital Code only have a payoff of 1 euro if we are in the money and 0 else. And here since we are talking about a very big K of 650000$ then when a european call get in the money even if it gets slightly above 650000 it might still be a very big amount of money like few thousands euros, and so to compensate we need a lot of Digital European Call giving a 1$ payoff. But our scenario isn't realistic because it isn't possible that such an extreme event happen with an asset that reach from a 53.6$ values up to a 650000$ value.

3 Acceptance-Rejection

1. Show that the ratio in the previous exercise can be expressed as a conditional expectation $\mathbb{E}[\Phi(X) \mid X \geq A]$ where $A \in \mathbb{R}$, and $X \sim \mathcal{N}(0,1)$.

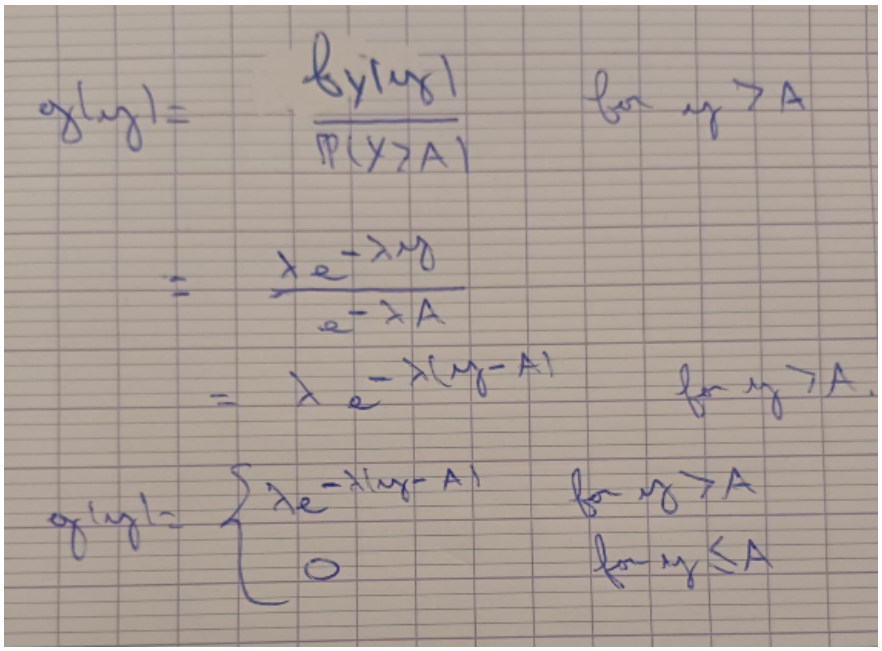


2. Let $A > 0$. If $Y \sim \mathcal{E}(\lambda)$ with $\lambda > 0$, what is the conditional distribution of $Y - A$ given $Y \geq A$? Name it.



We recognize the distribution law of an exponential, $\mathcal{E}(\lambda)$, with $P(Y > t) = e^{-\lambda t}$

3. We denote by g the density of $Y \mid Y > A$. Give the expression of g. We know that the density of an exponential distribution is : $f_Y(y) = \lambda e^{-\lambda y}$, and the probability $P(Y > A) = e^{-\lambda A}$.



4. Propose an algorithm to generate $Y \mid Y > A$.

As shown in question 2, X=Y-A follows an exponantial distribution.
And we also have Y=X+A
Algorithm :
We generate an uniform Law, because we will use it to generate an exponantial law.

$$U \sim U(0,1)$$

Then we generate the exponantial law X :

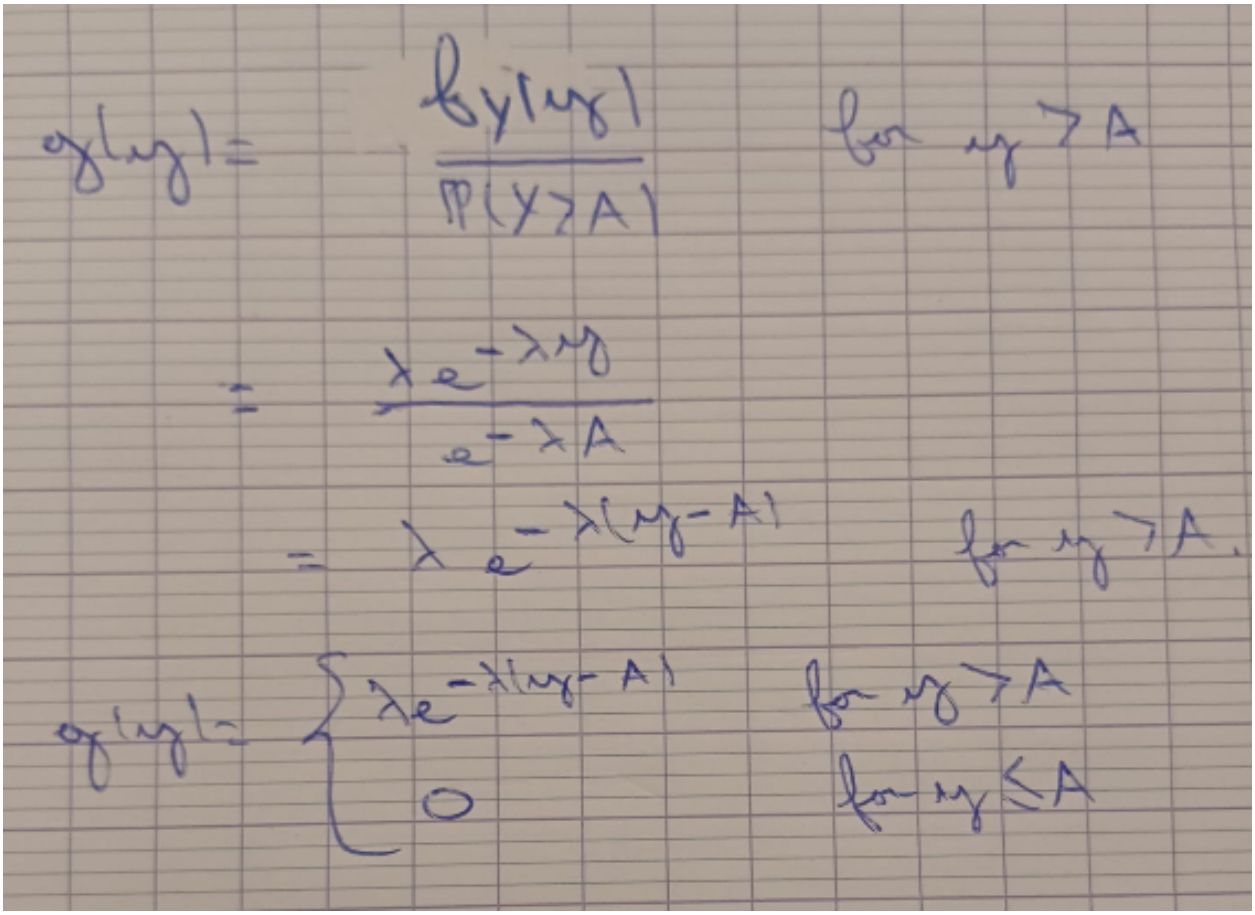
$$X = -\frac{1}{\lambda}\ln(U)$$

Then we add the shift A :

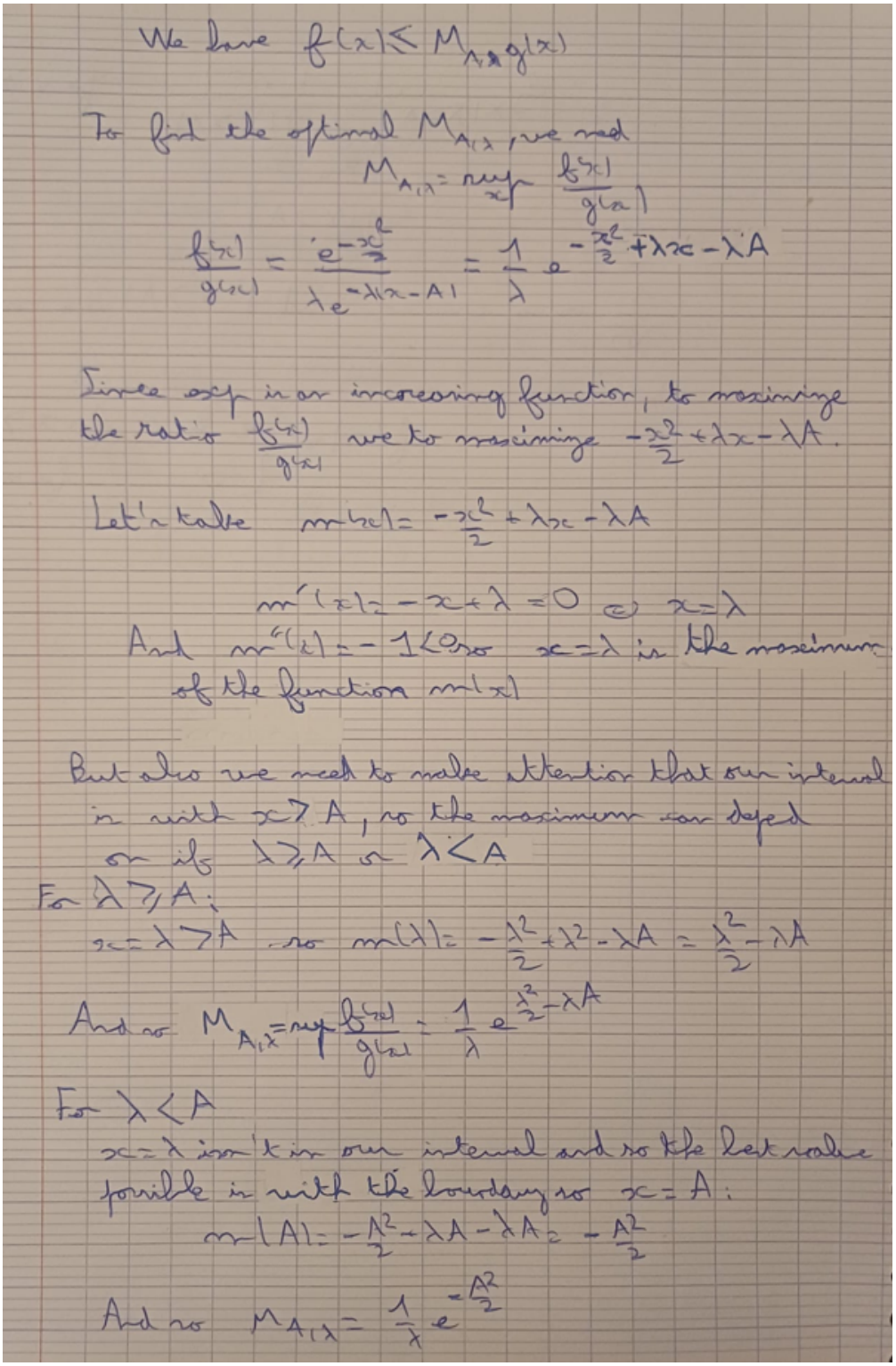
$$Y = X + A$$

And finally we return Y

5. Let $X \sim \mathcal{N}(0,1)$. Write the expression of $f_{X|X>A}$ up to a normalizing constant (the simplest one), let us denote this function by f.

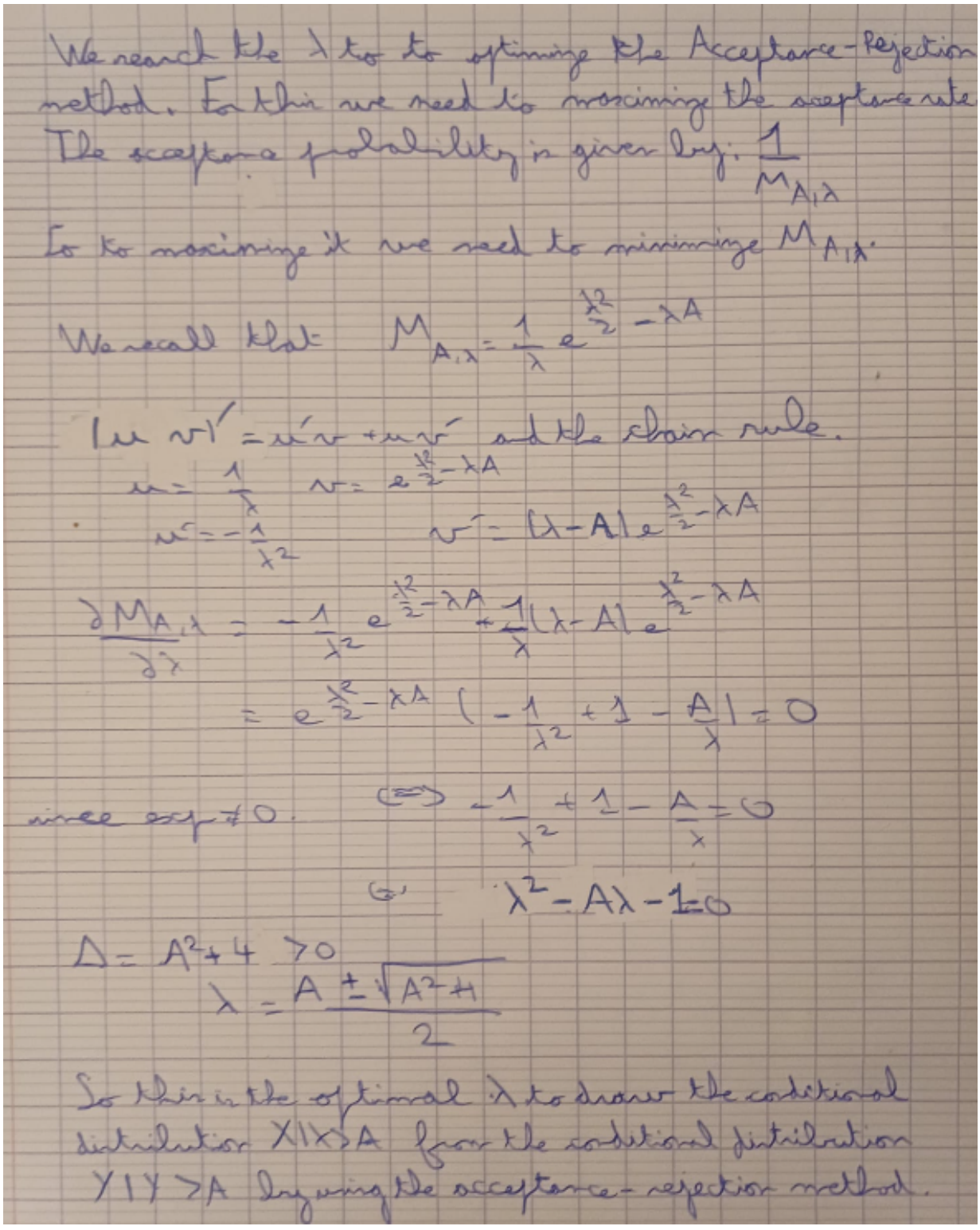


6. Find the best constant $M_{A,\lambda}$ such that $\forall x \in \mathbb{R}$, $f(x) \leq M_{A,\lambda} g(x)$.



7. How to choose λ depending on the value of A to draw the conditional distribution $X \mid X > A$ from the conditional distribution $Y \mid Y > A$ by using the

acceptance-rejection method?



8. Implement the algorithm and compute the conditional expectation in question 1.

Acceptance-Rejection Method

Sample Y with density g.
Generate $U \sim \mathcal{U}([0,1])$ that is independent from Y.
If $U \leq \frac{f(Y)}{Mg(Y)}$, then set $X = Y$ ("acceptance"); otherwise go back to step 1 ("rejection").

```
A = (np.log(K/S0)-(r-sigma**2/2)*T)/(sigma*np.sqrt(T)) #-d2

lamb = (A + np.sqrt(A**2 + 4)) / 2
N=100000

samples = []

while len(samples)<N:
    U1 = np.random.rand(N)
    Y = A-(1/lamb)*np.log(U1)

    U2 = np.random.rand(N) #U2 is the uniform for the acceptance
    #condition

    acceptance_prob = np.exp(-0.5*(Y-lamb)**2)

    accepted=Y[U2<= acceptance_prob]
    samples.extend(accepted)

samples = np.array(samples)


S_T = S0*np.exp((r-sigma**2/2)*T+sigma*np.sqrt(T)*samples)
Phi_X = S_T-K
#the cond expectation E[Phi(X) | X >= A]
expected_value = np.mean(Phi_X)
```

4 Quasi-Monte Carlo

1. Given a b-ary expansion $(a_j(k))$ of k, write a function that computes the b-ary expansion of $k+1$ WITHOUT evaluating k using the expansion (e.g., for $b = 10$: $(9,9,2,1) \mapsto (0,0,3,1)$). What is the complexity in k in the worst case?

The b-ary expansion, which is an incrementation of 1 of the number but put in a special base. In the example given in base b=10 : (9, 9, 2, 1), it represents the number 1299, so an increment of 1 would be 1300 and so it would then be (0,0,3,1). But as specified in the assignment we can't do it like this, by evaluating the value adding and converting it. We need to directly do the computation on the list of digits. For this we add 1 to the first digit

and return the new list, but if this digit is equal to the base-1 then we have to set it to 0 and to look at the next digit and add it 1, and repeat the process until you don't get a digit that was equal to base-1. Also you stop after going through all the digits. If this loop set all the values to 0 because they were all equal to base-1, example (9,9,9) would become (0,0,0,1). So if we get out of the loop we need to add a 1 at the end of the number expansion.

```
def increment_b_ary(digits, b):
    for i in range(len(digits)):
        if digits[i]<b-1:
            digits[i]+=1
            return digits
        else:
            digits[i]=0
    digits.append(1)
    return digits
```

For a value k, we loop on it's b-ary expansion, and a b-ary expansion can't be higher than the number of digits in the b-ary expansion, and the max it can loop on is the length, and it would be the case of all digits are equal to b-1. Now for the maximum length that a b-ary expansion could take based on a value k. When we look at a simple example (base b=10), we have that for k<9<b we have 1 digit, for k<99<b^2 we have 2 digits, for k<999<b^3 we have 3 digits. We can generalize this. With N being the number of digits (the length), We know that :

$$k < b^N$$

Actually this doesn't help us to majorate N, which is our goal so we will rather take :

$$k \geq b^{N-1}$$

And so by taking the log in base b we get :

$$\log_b(k) \geq \log_b(b^{N-1})$$

$$\log_b(k) \geq N - 1$$

$$N \leq \log_b(k) + 1$$

Because as we said above the maximum of iterations of the loop is equal to N. Then with this majoration we get the maximum number of loop is $log_b(k) + 1$, so we can conclude that the complexity in the worst case is $O(\log_b(k))$

2. Use the previous function to implement an algorithm that returns the k first terms of a Van der Corput sequence of base b. Horner's method must be used.

Here is what Horner's method look like, with d_0, d_1.. being the digits :

$$d_0 * x^0 + d_1 * x^1 + d_2 * x^2 + ... + d_n * x^n = d_0 + x(d_1 + x(d_2 + x(d_3 + ... + (d_{n-1} + xd_n)...))$$

And for us x is 1/b and so this gives us, but also for us since we want all the number to be below 1 to have them values as a 0.xxx we need to start not at b^0 but at b^1 :

$$\frac{d_0}{b^1} + \frac{d_1}{b^2} + \frac{d_2}{b^3} + ... + \frac{d_n}{b^{n+1}} = \frac{1}{b}(d_0 + \frac{1}{b}(d_1 + \frac{1}{b}(d_2 + \dots)))$$

And so we want the k first term of a Van der Corput sequence of base b, so we will start at 0 and then use the k-ary expansion to loop until we reach k. And express each of use using the Horner's model describe above.

```
def van_der_corput_horner_model(k, b):
    sequence = []
    digits = []
    for i in range(k):
        val = 0
        for d in range(len(digits)-1, -1, -1):
            val = (val+digits[d])/b
        sequence.append(val)
        digits = increment_b_ary(digits,b)
    return np.array(sequence)
```

3. Combine Exercise 2 with QMC to compute the ratio.

The only difference with Exercise 2 is that this time we won't generate the Standard Normal Law N(0,1) with Box Muller but we will use the Van Der Corput sequence, that is a list of values between 0 and 1 since the numbers are the 0.xxx with xxx being the digits. And then we use this values between 0 and 1 to take the inverse cumulative distribution of the standard normal law and so get out Z vector with N=100000 values, that then like before we drift with mu=-d2, we compute St, the payoff of European Call option and Digital Call option, the L_i, we choose an arbitrary C again. and finally compute the formula of the sum of the payoff of european call times the exponantial of $l_i + C$ and divided by the sum of the payoff of digital call times the exponantial of $l_i + C$

```
S0 = 53.6
r = 0.015
sigma = 0.235
T = 1
K = 650000
N = 100000
base = 2

d1 = (np.log(S0/K)+(r+sigma**2/2)*T)/(sigma*np.sqrt(T))
d2 = d1-sigma*np.sqrt(T)
mu = -d2

u = van_der_corput_horner_model(N, base)[1:] #here we do [1:] to
#drop the first value of the sequence because it is always 0 and
```

```
#we can't take a ppf for 0 (or it is +/-infinity)
Z = norm.ppf(u)

X = mu+Z
St = S0*np.exp((r-sigma**2/2)*T+sigma*np.sqrt(T)*X)

payoff_C = np.maximum(St-K,0)
payoff_D = (St>K).astype(float)

l_i = -mu*X+0.5*mu**2
C=1000 # this value is arbitrary and can be changed but need to be
#high enought to don t have exponantial of -745 and below or
#not too high too don t have a value that can t be computed .
ratio=np.sum(payoff_C*np.exp(l_i+C))/np.sum(payoff_D*np.exp(l_i+C))
```

By doing this we found once again the same results, with a european call option being worth a little bit above 3800 european digital call options. Also this time since a sequence is constant we always get the same results, but if we change the base and the number of estimations then it does change a little bit the results.

4. Compute the associated standard error given by randomized QMC.

So currently as said before there is no variance because the sequence are deterministic and stay the same. So we will now use the QMC randomized method and compute the associated standard error. For QMC the goal is to shift the random uniform law by an other random uniform law and to take it's modulus to make sure it stays below 1. And we then do the exact same thing to find the ratios. We get generate several ratios and we then compute the standard deviation of the ratios generated.

One way to do it: given a sequence $(u_k)_{k\leq n}$, we shift it randomly by:

$$u_k^i = u_k + v_i \pmod 1$$

where (v_i) is an i.i.d. sequence distributed as $\mathcal{U}([0,1])$.

```
def qmc_ratio_estimate(u_base, S0, K, sigma, r, T, mu):
    v = np.random.rand() #generate the random shift

    u_shifted = (u_base+v) % 1

    Z = norm.ppf(u_shifted)

    X = mu+Z
    St = S0*np.exp((r-sigma**2/2)*T+sigma*np.sqrt(T)*X)

    payoff_C = np.maximum(St-K,0)
    payoff_D = (St>K).astype(float)
```

```
    l_i = -mu*X+0.5*mu**2
    C=1000
    ratio = np.sum(payoff_C*np.exp(l_i+C))/np.sum(payoff_D*np.exp(l_i+C))
    return ratio

u_base = van_der_corput_horner_model(N, base)
m=100

d1 = (np.log(S0/K)+(r+sigma**2/2)*T)/(sigma*np.sqrt(T))
d2 = d1-sigma*np.sqrt(T)
mu = -d2

ratios = []
for i in range(m):
    ratios.append(qmc_ratio_estimate(u_base, S0, K, sigma, r, T, mu))

ratios = np.array(ratios)
std_error = np.std(ratios, ddof=1)/np.sqrt(m)
```

With this RQMC method on the Van Der Corput sequence, we get a standard error deviation of around 0.22.

5 Some Data

1. Make summary statistics of the assets: drift, volatility, log-return correlation, along with relevant plots.

We are given a CSV called data_simulation_methods. This csv contains the daily returns of two assets. To calculate the metrics we first need to compute the log return of the prices like suggested in the assignment. Here are the formulas for the different required metrics :

The log returns are equal to :

$$R_t = \ln\left(\frac{S_t}{S_{t-1}}\right)$$

Now to find the drift, First of all let's recall the SDE of the asset prices, that is the one of a Geometric Brownian motion,

$$dS_t = \mu S_t dt + \sigma S_t dW_t$$

So we need to find μ. We then can use Itô formula with f(x)=ln(x) since S_t is an Itô process and f(x) is C^2 we get :

$$d(\ln S_t) = (\mu - \frac{\sigma^2}{2})dt + \sigma dW_t$$

The drift term in the log of the prices is $\mu - \frac{\sigma^2}{2}$, which is different than the drift we want that was just μ. So when we take the expectation of log returns, if we want the drift we also

need to add a term $+\frac{\sigma^2}{2}$ to get back the μ, in fact this is just a formula we can obtain by using the CAGR that is $CAGR \approx \mu - \frac{\sigma^2}{2}$.

And so the formula for the drift is :

$$\mu_{annual} = \text{mean}(R_t) \times 252 + \frac{\sigma^2_{annual}}{2}$$

For the volatility it is just the daily vol times the sqrt of the number of trading days, 252.

$$\sigma^2_{annual} = \sqrt{252}\sigma^2$$

For the correlations we will just use the corr() function that is incorporated in python, but we will still recall the formula for it :

$$\rho = \frac{\sum_{i=1}^{n}(R_{asset1_i} - \bar{R_{asset1}})(R_{Asset2_i} - \bar{R_{Asset2}})}{\sqrt{\sum_{i=1}^{n}(R_{asset1_i} - \bar{R_{asset1}})^2}\sqrt{\sum_{i=1}^{n}(R_{Asset2_i} - \bar{R_{Asset2}})^2}}$$

with $\bar{R_{asset1}}$ being the average of R_{asset1} and $\bar{R_{asset2}}$ being the average of R_{asset2}

```
log_returns = np.log(df / df.shift(1)).dropna()

daily_mean = log_returns.mean()
daily_vol = log_returns.std()
correlation_mat = log_returns.corr()
correlation=correlation_mat.iloc[0, 1]

trading_days = 252
ann_vol = daily_vol*np.sqrt(trading_days)

ann_drift = daily_mean*trading_days+(ann_vol**2)/2
```

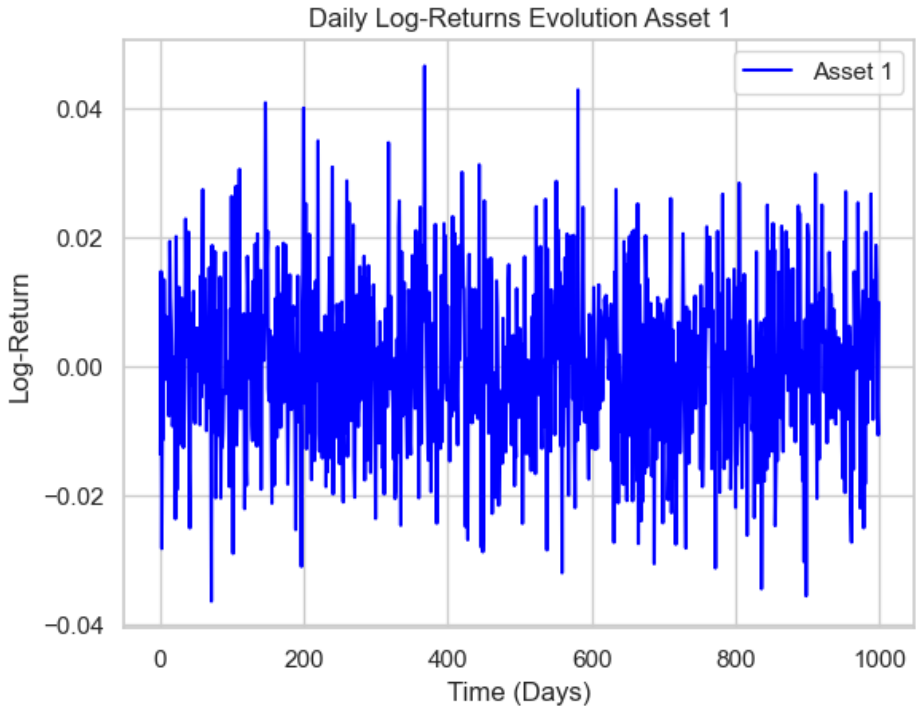
Metrics	**Asset 1**	**Asset 2**
Daily volatility	0.012706	0.012666
Annualized volatility	0.201708	0.201070
Annualized drift	0.057037	0.012300
Log_return corr	0.72019	0.72019

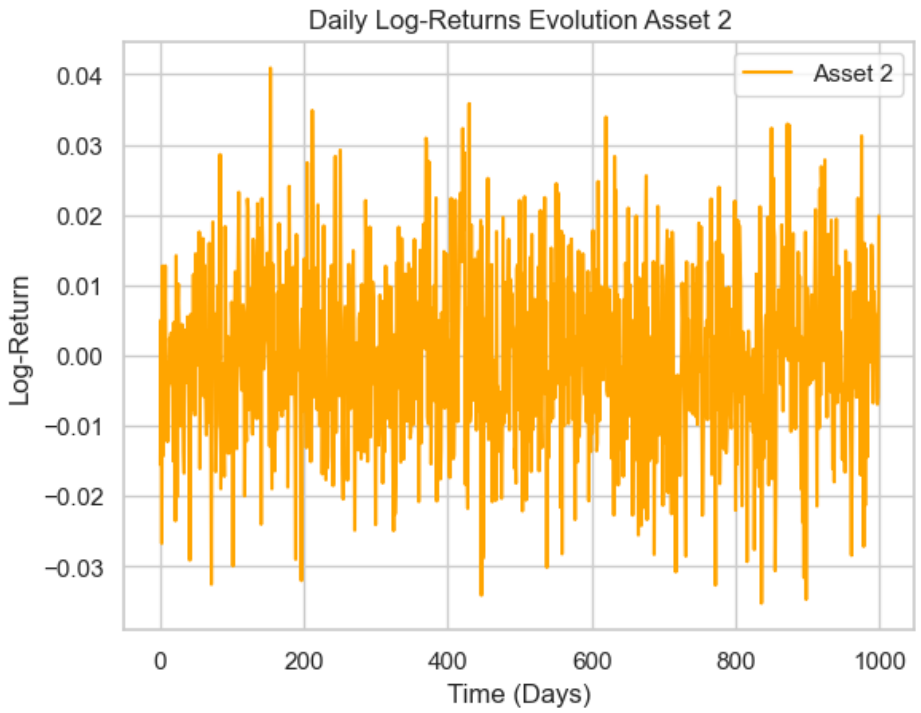
We first plotted the evolution of the price to see what really happens to the asset during the time we will study it. We can see that both graphs looks pretty similar which is in agreement with the correlation we found of 0.72 that is pretty high. Also we can note that the prices of asset 2 are higher than asset 1 with asset 1 that stay around 60 and asset 2 around 160.





Then to check for stationary, we plotted the Log returns evolution of asset 1 and asset 2. Indeed when we look at the plots it looks very stationary and stays around 0.






```
plt.figure()
plt.plot(df.index, df['Asset_1'], color='blue')
plt.title('Asset 1 Price Evolution')
plt.xlabel('Time (Days)')
```

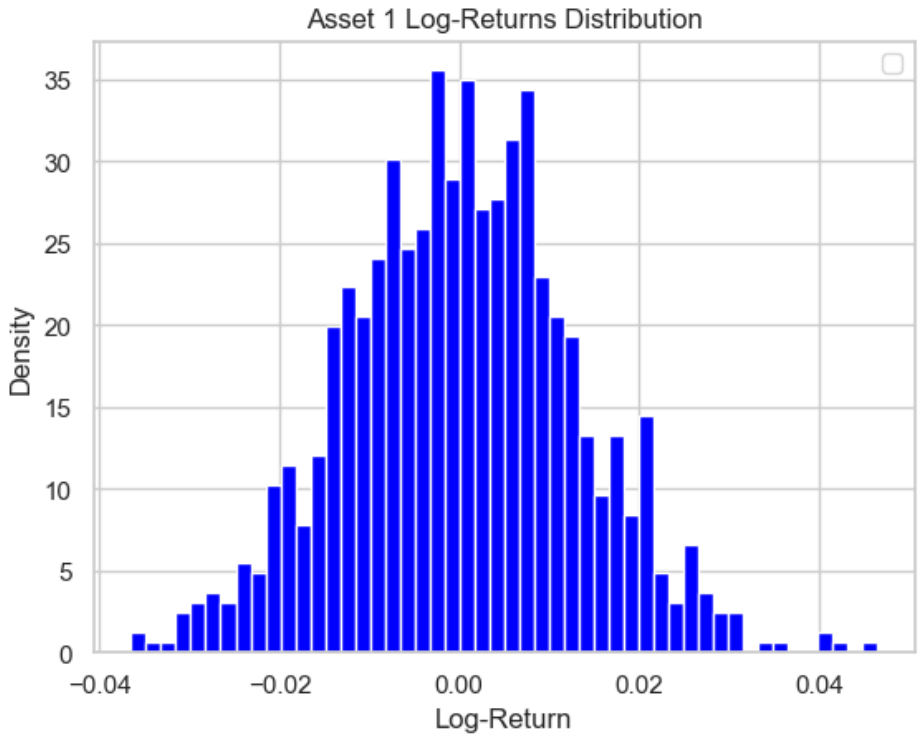
```
plt.ylabel('Price')
plt.show()

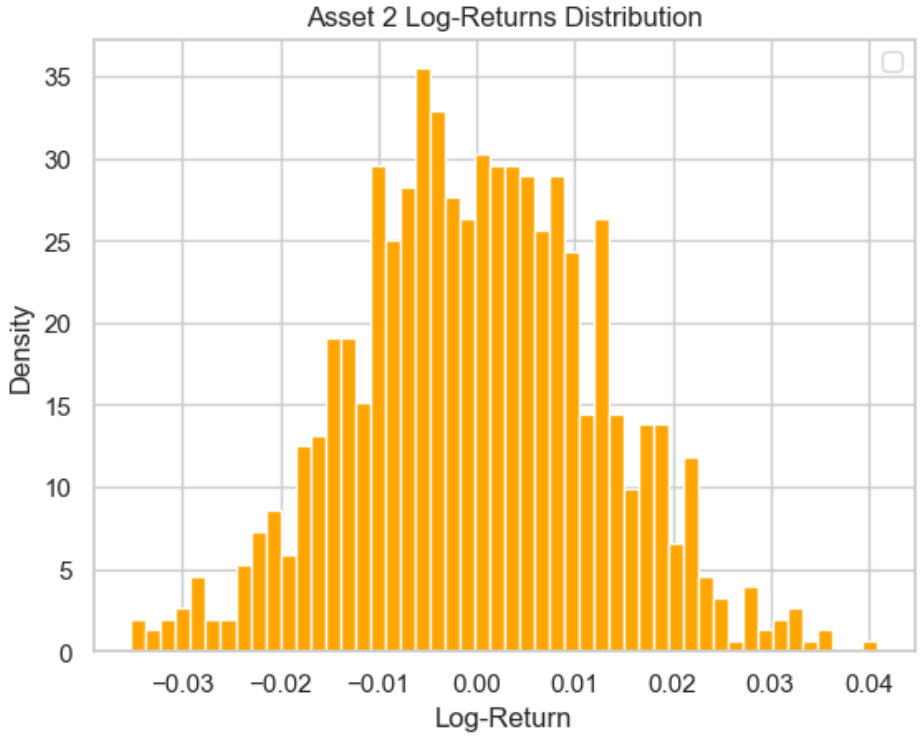
plt.figure()
plt.plot(df.index, df['Asset_2'], color='orange')
plt.title('Asset 2 Price Evolution')
plt.xlabel('Time (Days)')
plt.ylabel('Price')
plt.show()

plt.figure()
plt.plot(log_returns.index, log_returns['Asset_1'], label='Asset 1', color='blue')
plt.title('Daily Log-Returns Evolution Asset 1')
plt.xlabel('Time (Days)')
plt.ylabel('Log-Return')
plt.legend()
plt.show()

plt.figure()
plt.plot(log_returns.index, log_returns['Asset_2'], label='Asset 2', color='orange')
plt.title('Daily Log-Returns Evolution Asset 2')
plt.xlabel('Time (Days)')
plt.ylabel('Log-Return')
plt.legend()
plt.show()
```

2. Plot the histograms of the log-returns of the assets, is a Gaussian model realistic to model the log-returns' marginal distributions?





This 2 plots both looks like a normal law, they are very concentrated on the middle, with very little amount of extreme values, they have a bell curve look a like density function. But

they don't look like perfect gaussian distribution thought. I would still say it can be realistic to model it but it wouldn't be perfect.

3. Make a scatter plot of the log-returns of the 2 assets, is a bivariate Gaussian model realistic to describe their joint distribution?



No, this plot shows that a bivariate gaussian wouldn't be appropriate to do a realistic model of the joint distribution. A gaussian bivariate should be symmetrical between it's top right with it's bottom left. But here the bottom are very tight but at the top is very wide and it take the form of a cone. his shows that when Asset 1 get high negative return, Asset 2 almost always crashes with it. The dots are much more spread out and diffuse. They don't form that same tight, extreme directional tail. Because the assets are clearly more correlated during market drops than during market rallies, a Gaussian model (which assumes the correlation is identical everywhere) will fail here. A bivariate gaussian generates a plot of the form of an oval, meanwhile here it is clearly more like a code. The bottom start like an oval but after going to the top it widden. And finally there are some very far outliers when looking at the extreme gains top right of the plot.

```
plt.figure()
plt.scatter(log_returns['Asset_1'], log_returns['Asset_2'])
plt.title('Log−Returns Scatter Plot')
plt.xlabel('Asset 1 Returns')
plt.ylabel('Asset 2 Returns')
plt.show()
```

4. Compute Kendall's rank correlation coefficient on the log-returns, then fit a Clayton copula on the data.

To compute the Kendall rank correlation coefficient we use the standard estimator of Kendall's tau, with the formula given in the course :

$$R_\tau = \frac{1}{\binom{n}{2}} \sum_{1 \le k < l \le n} \operatorname{sgn}\left(\left(X_k^1 - X_l^1\right)\left(X_k^2 - X_l^2\right)\right)$$

So we need to differentiate the X_1 with other X_1, while respecting that l is always bigger than k. And same for the X_2. Then we multiply each day and apply the sign function, if positive it give a 1 if negative a -1 and if null then 0. We then sum every single sign. After this we divide by the 2 out of n which is the number of possible combinations to get the average. Then for the theta of the Clayton copula, we are given from the course that,

$$\tau = \frac{\theta}{\theta + 2}$$

Then we isolate τ and we get that theta is equal to :

$$\frac{2\tau}{1 - \tau} = \theta$$

The results we got are τ=0.5525 And θ=2.46917

A value of 0.5525 indicates a strong, positive rank correlation between the assets.

```
X1 = log_returns['Asset_1'].values
X2 = log_returns['Asset_2'].values
n = len(X1)

combinations = (n*(n-1))/2 #2 out of n observations

tau_sum = 0

for k in range(n):
    for l in range(k+1,n):

        diff_X1 = X1[k] - X1[l]
        diff_X2 = X2[k] - X2[l]

        pair_sign = np.sign(diff_X1*diff_X2)

        tau_sum += pair_sign

kendall_tau = tau_sum / combinations

theta = (2*kendall_tau)/(1-kendall_tau)
```

5. Simulate the 1000 next days' data for the 2 price trajectories, compare the summary statistics + plots of the previous questions with those of the original data.

We will simulate the data by using the fitted Clayton Copula that we got in previous question with $\theta = 2.46917$, We start by taking this formula :

$$C_\theta^{\mathrm{Cl}}(u_1, u_2) = \left(u_1^{-\theta} + u_2^{-\theta} - 1\right)^{-1/\theta}, \quad 0 \leq \theta < \infty$$

The goal is to first generate u1 and then generate u2 thanks to this generated u1. So we need to isolate u1.

Sample $U_1 \sim \mathcal{U}([0,1])$, we obtain the value u_1. Sample U_2 consistent with U_1:

$$P[U_2 \leq u_2 | U_1 = u_1] = \frac{\partial_1 C(u_1, u_2, 1, \ldots, 1)}{\partial_1 C(u_1, 1, 1, \ldots, 1)}$$

This conditional CDF is to be used in the inverse transform method. The denominator $\partial_1 C(u_1, 1)$ is just the derivative of the marginal distribution. Since the marginal of a copula is a uniform distribution $C(u_1, 1) = u_1$ And so we are left with just

$$\frac{\partial C(u_1, u_2)}{\partial u_1}$$

Now let's compute the derivative.

A copula is a joint cumulative probability function
So:

$$C(u_1,u_2) = \mathbb{P}(U_1 \leq u_1 \cap U_2 \leq u_2)$$

And for $C(u_1,1) = \mathbb{P}(U_1 \leq u_1 \cap U_2 \leq 1)$
and $U_2 \in [0,1]$ so $U_2 \leq 1$ always true
So $C(u_1,1) = \mathbb{P}(U_1 \leq 1)$
$= u_1$

So $$\frac{\partial C(u_1,u_2)}{\partial C(u_1,1)} = \frac{\partial C(u_1,u_2)}{\partial u_1}$$

And $C(u_1,u_2) = (u_1^{-\theta} + u_2^{-\theta} - 1)^{-\frac{1}{\theta}}$
by the definition of the bivariate Clayton copula.

$$\frac{\partial C(u_1,u_2)}{\partial u_1} = -\frac{1}{\theta} u^{-\frac{1}{\theta}-1}$$

$$= -\frac{1}{\theta} \times (-\theta u_1^{-\theta-1}) (u_1^{-\theta} + u_2^{-\theta} - 1)^{-\frac{1}{\theta}-1}$$

$$= u_1^{-\theta-1} (u_1^{-\theta} + u_2^{-\theta} - 1)^{-\frac{1}{\theta}-1}$$

Let's note $U_c \sim U([0,1])$ that is the random law distribution of $\frac{\partial C(u_1,u_2)}{\partial C(u_1,1)}$

& we have $u_c = u_1^{-\theta-1} (u_1^{-\theta} + u_2^{-\theta} - 1)^{-\frac{1}{\theta}-1}$

$$\Leftrightarrow u_c u_1^{\theta+1} = (u_1^{-\theta} + u_2^{-\theta} - 1)^{-\frac{\theta+1}{\theta}}$$

$$\Leftrightarrow u_c^{\frac{\theta}{\theta+1}} u_1^{-\theta} = u_1^{-\theta} + u_2^{-\theta} - 1$$

$$\Leftrightarrow u_2 = (u_c^{\frac{\theta}{\theta+1}} u_1^{-\theta} + 1 - u_1^{-\theta})^{-\frac{1}{\theta}}$$

Now we have the 2 U_1 and U_2 generated. we apply the inverse normal distribution and get a Z_1 and Z_2, two standard normal law N(0,1). And so now we know that

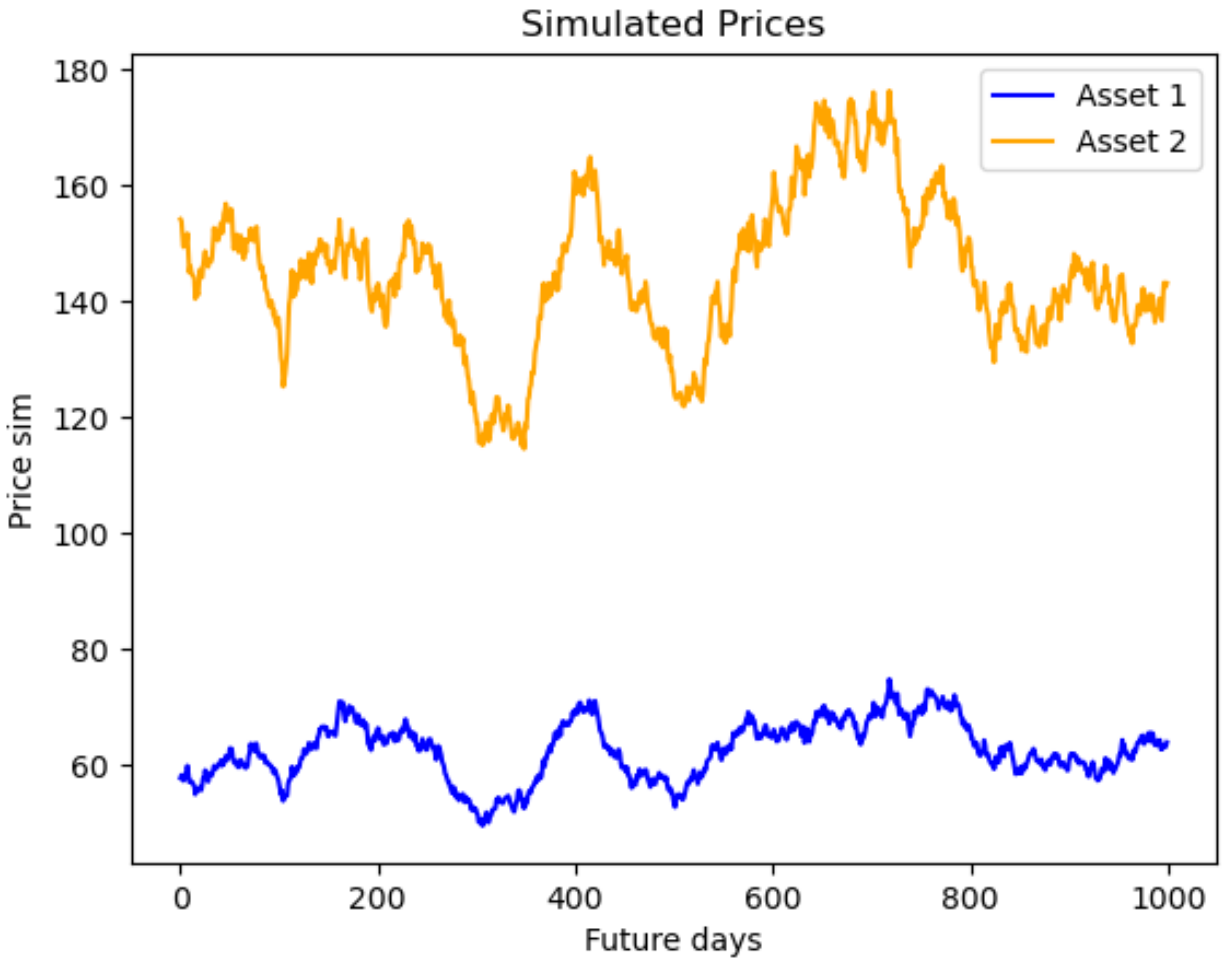
$$\mu + \sigma Z \sim \mathcal{N}(\mu, \sigma^2)$$

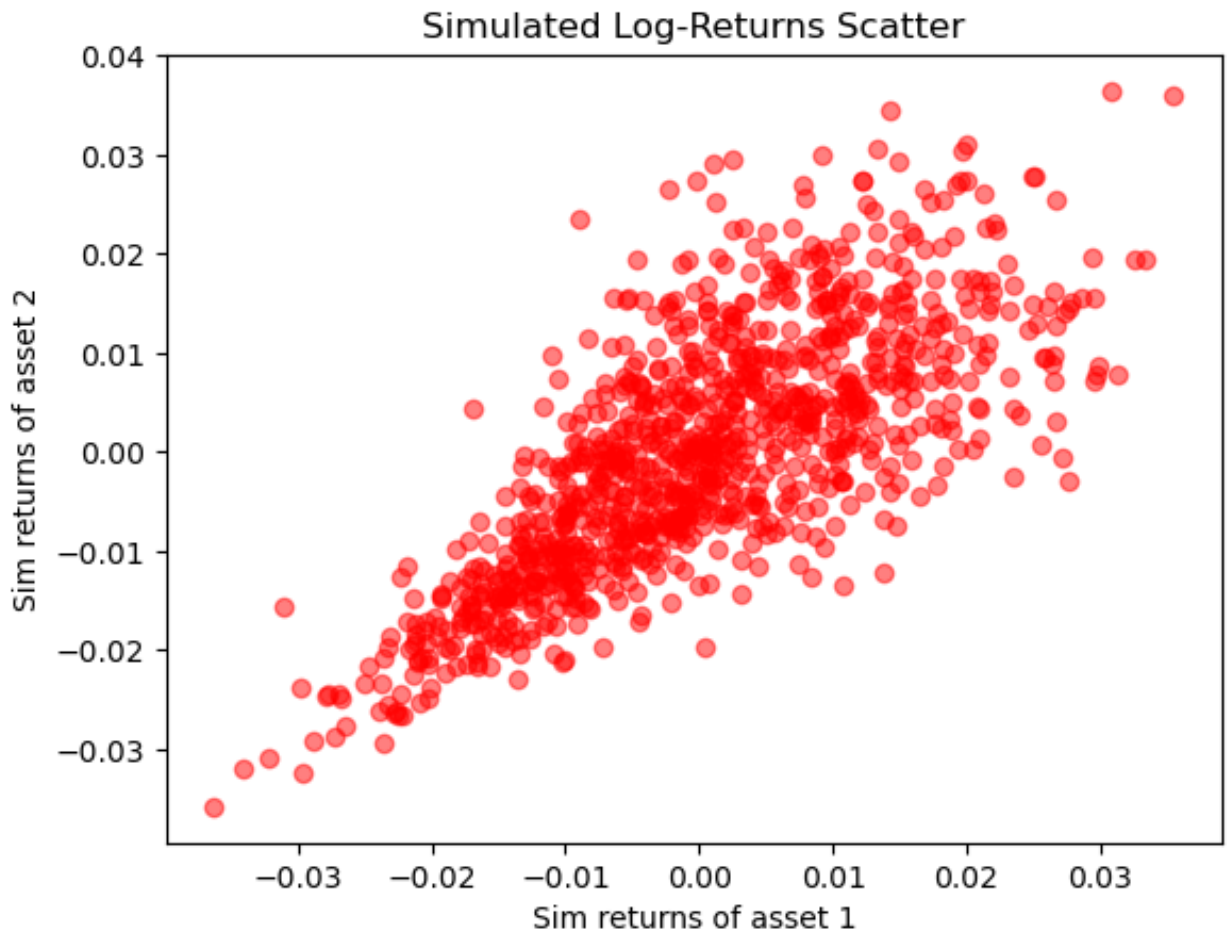
And we generated the law for our simulated returns. Then we need to simulate the evolution of the prices associated to this simulated returns. knowing that the price using the log returns is given by :

$$S_1 * \exp(\log_returns)$$

And so we start with the first log return and we iterate each day with the new log return on the new price that resulted from this equation.

Since the forecasting is done using random uniform variables the results of the mean and other metrics are changed each time along with the plot of the asset prices, meanwhile the scatter plot of there log returns are always keeping a same shape. So I'll put one of the results I got :





Metric	**Asset**	**Original Data**	**Simulated Data**
Daily Volatility	Asset 1	0.012706	0.012333
Daily Volatility	Asset 2	0.012666	0.012398
Annualized Volatility	Asset 1	0.201708	0.195781
Annualized Volatility	Asset 2	0.201070	0.196808
Daily Mean	Asset 1	0.000146	0.000084
Daily Mean	Asset 2	-0.000031	-0.000088
Annualized Drift	Asset 1	0.057037	0.040363
Annualized Drift	Asset 2	0.012300	-0.002755
Correlation	both	0.72019	0.752697

Comparing the metrics of our simulation with the original data, we can see that the volatility is very well reproduced with 20.17% and 20.11% for original data and 19.58% and 19.68% for the simulated data. The correlation also stayed pretty close with 0.72 for original data and 0.75 for simulated data. But the drift was slightly off with 5.7% and 1.2% for asset 1 and 2 of the original data against respectively 4% and -0.2%.

The results are satisfying because our simulation respected the structure of the two assets. With a correlation and volatility staying similar and the graph of the scatter plot. But the drift/mean changed because obviously it is a random simulation and there are only 1000 data points which isn't enought to have a good average. If we had done this much more times and averaged it, it probably would have given a similar drift than the original data, but in our current set up it won't give similar results.

```
theta = 2.46917
n = 1000

S1_0 = df['Asset_1'].tail(1).values[0]
S2_0 = df['Asset_2'].tail(1).values[0]
mu1 = daily_mean['Asset_1']
mu2 = daily_mean['Asset_2']
vol1 = daily_vol['Asset_1']
vol2 = daily_vol['Asset_2']

u1 = np.random.uniform(0,1,n)
u_c = np.random.uniform(0,1,n)

u2=(1-u1**(-theta)+u1**(-theta)*u_c**(-theta/(theta+1)))**(-1/theta)

Z1 = norm.ppf(u1)
Z2 = norm.ppf(u2)

sim_returns_1 = mu1 + vol1*Z1
sim_returns_2 = mu2 + vol2*Z2

sim_prices_1 = []
sim_prices_2 = []

sim_prices_1.append(S1_0*np.exp(sim_returns_1[0]))
sim_prices_2.append(S2_0*np.exp(sim_returns_2[0]))

for i in range(1, n):
    sim_prices_1.append(sim_prices_1[i-1] * np.exp(sim_returns_1[i]))
    sim_prices_2.append(sim_prices_2[i-1] * np.exp(sim_returns_2[i]))

sim_mean_1 = np.mean(sim_returns_1)
sim_mean_2 = np.mean(sim_returns_2)
sim_vol_1 = np.std(sim_returns_1)
sim_vol_2 = np.std(sim_returns_2)

sim_ann_drift_1 = (sim_mean_1+(sim_vol_1**2)/2)*252
sim_ann_drift_2 = (sim_mean_2+(sim_vol_2**2)/2)*252
```

```
sim_ann_vol_1 = sim_vol_1*np.sqrt(252)
sim_ann_vol_2 = sim_vol_2*np.sqrt(252)

sim_df = pd.DataFrame({ 'Asset 1': sim_returns_1, 'Asset 2': sim_returns_2 })
correlation_sim_mat = sim_df.corr()
correlation_sim=correlation_sim_mat.iloc[0, 1]

plt.figure()
plt.plot(sim_prices_1, label='Asset 1', color='blue')
plt.plot(sim_prices_2, label='Asset 2', color='orange')
plt.title('Simulated Prices')
plt.xlabel('Future days')
plt.ylabel('Price sim')
plt.legend()
plt.show()

plt.figure()
plt.scatter(sim_returns_1, sim_returns_2, color='red', alpha=0.5)
plt.title('Simulated Log-Returns Scatter')
plt.xlabel('Sim returns of asset 1')
plt.ylabel('Sim returns of asset 2')
plt.show()
```

6. Compare the Kendall's rank correlation and the Spearman's rank correlation between the original data and your simulated data using a statistical bootstrap.

I did the computation of the Kendall rank correlations is done with the same method as before so no need to explain again, can refer to prev questions. Now for the Spearman's rank correlation, we use this formula :

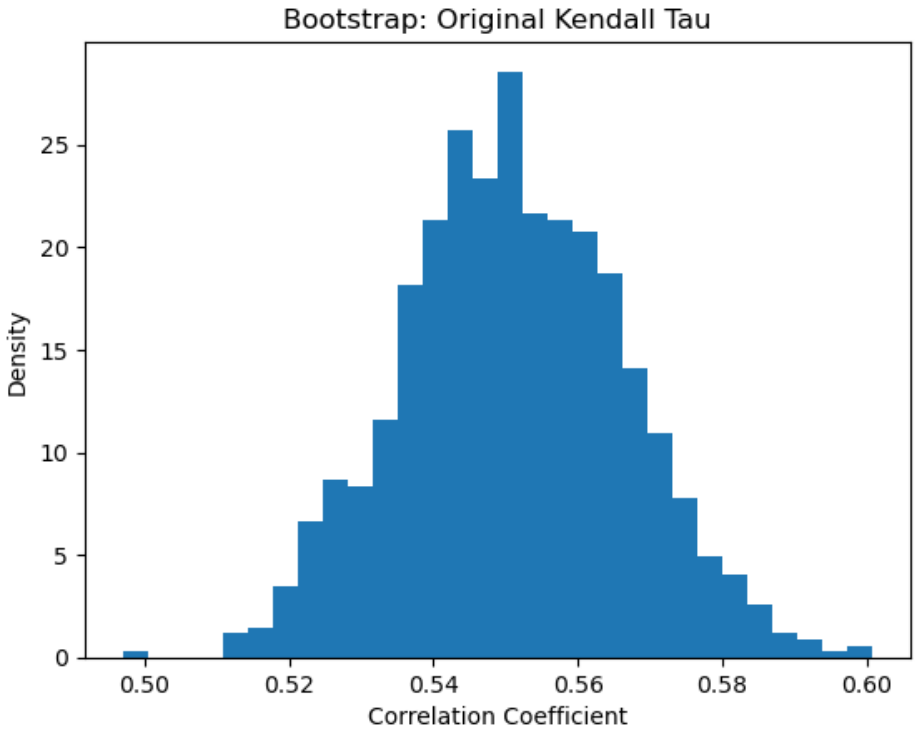
$$R_S = \frac{12}{n(n^2-1)} \sum_{k=1}^{n} \left(\text{rank}(X_k^1) - \frac{1}{2}(n+1) \right) \left(\text{rank}(X_k^2) - \frac{1}{2}(n+1) \right)$$

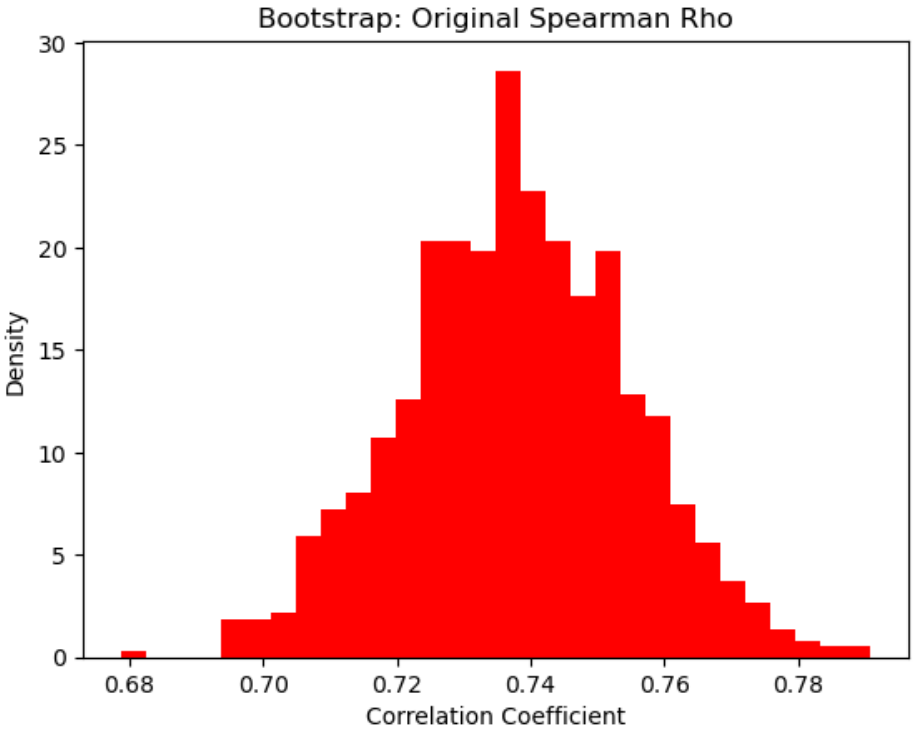
The hard part about this equation is to find the rank of the values of the X. For this we compared each values of X (the assets) by making it a column vector, and comparing it to the initial X and then we count how many time the value is above each of them and we add 1 to get the real rank because first rank is 1 not 0. The count is done by doing a sum of some True and False, and in this case the True are considered as 1 and False as 0.

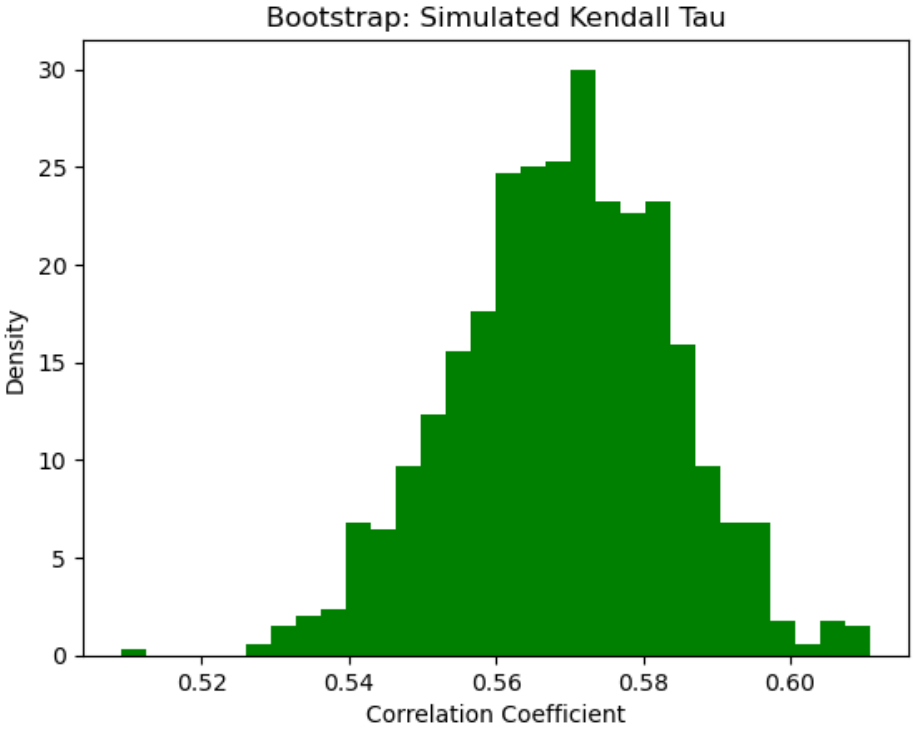
Now for bootstrap : The method is to do a loop of the number of simulation we want, we sample with replacement the data points from a limited data. The resampled data $\left(X_1^{\star(j)}, \ldots, X_n^{\star(j)}\right)$ are called bootstrap samples. We estimate our summary statistics (Kendall and Spearman) on each sample to generate a bootstrap distribution. Then we approximate the bootstrap distribution $S^\star$ of S. And then we take the mean and standard deviation of this 1000 generated rank correlations.

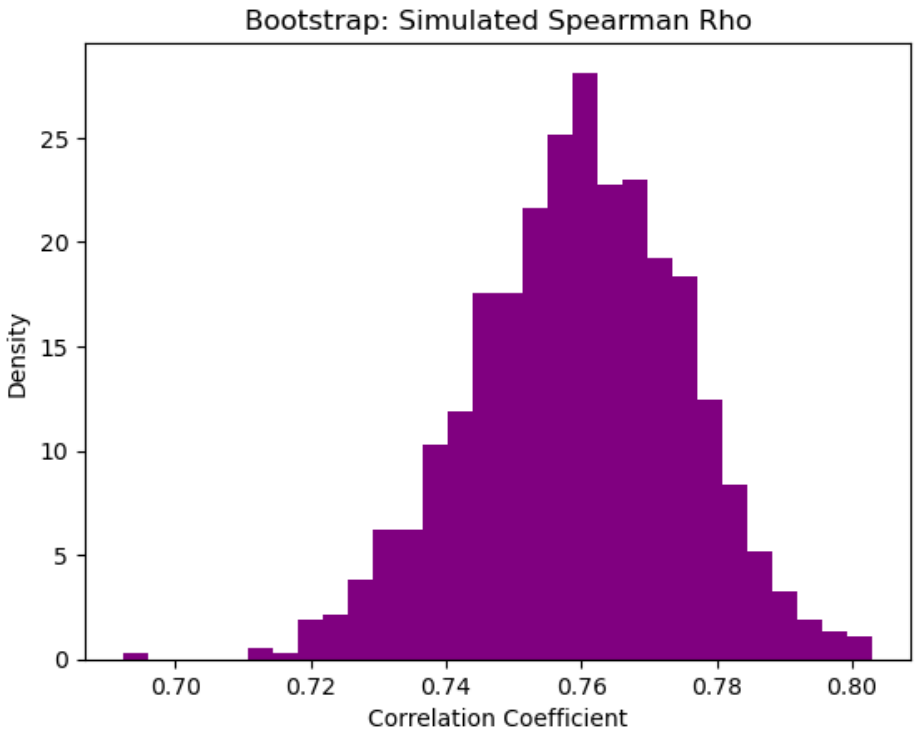
Metric	**Kendall τ**	**Spearman ρ**	**Sim Kendal τ**	**Sim Spearman ρ**
Mean	0.5518	0.7389	0.5692	0.7594
Standard Deviation	0.0162	0.0176	0.0.0147	0.0158
CI 95%	[0.5205, 0.5830]	[0.7052, 0.7721]	[0.5402, 0.5966]	[0.7273, 0.7888]

In both cases we can see that the Kendall τ is significantly lower than the Spearman ρ with a values of 0.5692 against 0.7594. Both metrics are in concordance that the 2 assets are positively correlated. When looking at the original values and the simulated values. Also we can note that the condifence interval of both Kendal τ and Spearman ρ are overlapping with there respective simulated values.




Bootstrap: Original Spearman Rho
Density
Correlation Coefficient
0
5
10
15
20
25
30
0.68
0.70
0.72
0.74
0.76
0.78


Bootstrap: Simulated Kendall Tau
Density
Correlation Coefficient
0
5
10
15
20
25
30
0.52
0.54
0.56
0.58
0.60




```
n_data = len(X1)
n_sim = len(sim_returns_1)

M = 1000

boot_kendall = []
boot_spearman = []
boot_kendall_sim = []
boot_spearman_sim = []

for b in range(M):

    u = np.random.uniform(0, 1, n_data)
    idx = (u*n_data).astype(int)

    x_1 = X1[idx]
    x_2 = X2[idx]
    #vectorisation of the prices. Instead of looping throught all pairs like in previous
    #question this time we do a vectorise operation to get the differences of all pairs at
    #once because the computation of the differences was very heavy and doing it 1000
    #times for a bootstrap would have took way too much
    #time and we be not efficient at all.
    #it would have been like 1 hours
    diff_x_1 = x_1[:, None] - x_1
```

```
    diff_x_2 = x_2[:, None] - x_2
    tau = np.sum(np.sign(diff_x_1)*np.sign(diff_x_2))/(n_data*(n_data-1))
    boot_kendall.append(tau)

    #so here we are selecting each element of x_1 and comparing it to all the other
    #elements of x_1 to get the values above it and count them and then we add 1 to get
    #the real rank because first rank is 1 not 0
    rank_x_1=np.sum(x_1[:,None]>x_1, axis=1)+1
    rank_x_2 = np.sum(x_2[:,None]> x_2, axis=1)+1
    term_x_1 = rank_x_1-0.5*(n_data+1)
    term_x_2 = rank_x_2-0.5*(n_data+1)
    rho = (12/(n_data*(n_data**2-1)))*np.sum(term_x_1*term_x_2)
    boot_spearman.append(rho)

    u_sim = np.random.uniform(0, 1, n_sim)
    idx_sim = (u_sim*n_sim).astype(int)

    x_1_sim = sim_returns_1[idx_sim]
    X_2_sim = sim_returns_2[idx_sim]

    diff_x_1_sim = x_1_sim[:, None]-x_1_sim
    diff_x_2_sim = X_2_sim[:, None]-X_2_sim
    tau_sim = np.sum(np.sign(diff_x_1_sim)*np.sign(diff_x_2_sim))/(n_sim*(n_sim-1))
    boot_kendall_sim.append(tau_sim)

    rank_x_1_sim = np.sum(x_1_sim[:, None]>x_1_sim, axis=1)+1
    rank_x_2_sim = np.sum(X_2_sim[:, None]>X_2_sim, axis=1)+1
    term_x_1_sim = rank_x_1_sim-0.5*(n_sim+1)
    term_x_2_sim = rank_x_2_sim-0.5*(n_sim+1)
    rho_sim = (12/(n_sim*(n_sim**2-1)))*np.sum(term_x_1_sim*term_x_2_sim)
    boot_spearman_sim.append(rho_sim)


boot_kendall_orig = np.array(boot_kendall_orig)
boot_spearman_orig = np.array(boot_spearman_orig)
boot_kendall_sim = np.array(boot_kendall_sim)
boot_spearman_sim = np.array(boot_spearman_sim)

mean_kendall=np.mean(boot_kendall)
std_kendall=np.std(boot_kendall)
ci_kendall = [np.percentile(boot_kendall, 2.5), np.percentile(boot_kendall, 97.5)]

mean_kendall_sim=np.mean(boot_kendall_sim)
std_kendall_sim=np.std(boot_kendall_sim)
ci_kendall_sim = [np.percentile(boot_kendall_sim, 2.5), np.percentile(boot_kendall_sim, 97.5)]
```

```
mean_spearman=np.mean(boot_spearman)
std_spearman=np.std(boot_spearman)
ci_spearman = [np.percentile(boot_spearman, 2.5), np.percentile(boot_spearman, 97.5)]

mean_spearman_sim=np.mean(boot_spearman_sim)
std_spearman_sim=np.std(boot_spearman_sim)
ci_spearman_sim=[np.percentile(boot_spearman_sim, 2.5),np.percentile(boot_spearman_sim, 97.5)]
```

```
plt.figure()
plt.plot()
plt.hist(boot_kendall_orig, bins=30, density=True)
plt.title("Bootstrap: Original Kendall Tau")
plt.xlabel("Correlation Coefficient")
plt.ylabel("Density")
plt.show()

plt.figure()
plt.plot()
plt.hist(boot_spearman_orig, bins=30,color="red", density=True)
plt.title("Bootstrap: Original Spearman Rho")
plt.xlabel("Correlation Coefficient")
plt.ylabel("Density")
plt.show()

plt.figure()
plt.plot()
plt.hist(boot_kendall_sim, bins=30,color="green" ,density=True)
plt.title("Bootstrap: Simulated Kendall Tau")
plt.xlabel("Correlation Coefficient")
plt.ylabel("Density")
plt.show()

plt.figure()
plt.plot()
plt.hist(boot_spearman_sim, bins=30,color="purple" ,density=True)
plt.title("Bootstrap: Simulated Spearman Rho")
plt.xlabel("Correlation Coefficient")
plt.ylabel("Density")
plt.show()
```